

Blueprint Arquitectónico de UI/UX para DBCWorkspace: Entorno de Entrenamiento ICPC de Alta Densidad

Introducción y Filosofía Fundamental de la Experiencia de Usuario

El diseño y desarrollo de DBCWorkspace plantea un desafío arquitectónico y de experiencia de usuario de extrema precisión, dado que se trata de una herramienta privada de entrenamiento algorítmico concebida exclusivamente para un equipo de la competición ICPC (International Collegiate Programming Contest) conformado por tres personas: un administrador y dos miembros adicionales. La restricción del público objetivo a un micro-equipo transforma radicalmente los paradigmas tradicionales del desarrollo de software. A diferencia de las aplicaciones comerciales o plataformas públicas de programación competitiva que deben priorizar el marketing visual, la incorporación guiada de nuevos usuarios (onboarding) y la retención mediante gamificación superficial, DBCWorkspace debe operar como una extensión directa y sin fricciones del flujo cognitivo de sus tres únicos usuarios. Según los lineamientos establecidos por la norma ISO 9241-210, la experiencia de usuario (UX) engloba todas las emociones, creencias, preferencias y comportamientos del individuo antes, durante y después del uso de un sistema.¹ En el contexto del entrenamiento de alto rendimiento algorítmico, donde la precisión matemática y la velocidad de resolución son críticas, cualquier distracción visual o ineficiencia en la interfaz se traduce en un aumento de la carga cognitiva y, por ende, en una degradación del desempeño del equipo.

La disciplina de la experiencia de usuario es inherentemente multidisciplinaria, convergiendo el diseño de interacción, la arquitectura de la información y la psicología cognitiva para estructurar lógicas que minimicen el esfuerzo del usuario.¹ Aplicando el modelo del panel de abeja (Honeycomb) conceptualizado por Peter Morville, pionero en arquitectura de la información, el diseño de DBCWorkspace debe maximizar tres facetas específicas: debe ser profundamente "Útil" (resolviendo el problema real de documentar y organizar el entrenamiento), "Encontrable" (permitiendo navegar sin perderse entre cientos de problemas algorítmicos) y "Usable" (garantizando facilidad de uso y minimizando la fricción).¹ En la práctica, el objetivo principal del diseño de software se traduce en reducir la fricción entre la intención del usuario y la respuesta del sistema, garantizando que la interacción produzca un resultado eficaz y eficiente con el menor esfuerzo posible.¹

Para materializar esta visión, la pila tecnológica seleccionada (el "stack") proporciona los cimientos técnicos ideales para una aplicación de respuesta inmediata y alta disponibilidad. La utilización de Next.js 14 bajo el paradigma del App Router permite un manejo eficiente del renderizado del lado del servidor (SSR) y componentes del lado del cliente, garantizando tiempos de carga ínfimos. La persistencia de datos se orquesta mediante PostgreSQL alojado

en la infraestructura de Neon, interactuando con la aplicación a través del ORM Prisma para asegurar la seguridad de tipos (type-safety) en todo el flujo de información. La seguridad y gestión de sesiones recae sobre Auth.js implementando un modelo de JSON Web Tokens (JWT), mientras que la construcción de la interfaz gráfica se delega a Tailwind CSS, lo que permite un control granular de cada píxel sin el peso de hojas de estilo monolíticas. La síntesis de estas tecnologías debe ser moldeada bajo una estética de diseño oscura y densa, inspirada directamente en la austeridad funcional de plataformas como AtCoder, la densidad de información de GitHub y el pragmatismo intelectual de Obsidian.²

El presente informe detalla exhaustivamente las especificaciones de interfaz de usuario (UI), las estrategias de implementación técnica y los modelos estructurales de datos requeridos para construir DBCWorkspace. El análisis aborda desde la teoría del color y la tipografía hasta la gestión del estado de la URL, el diseño de tablas de alta densidad, la integración de editores matemáticos avanzados y la arquitectura de un sistema de autenticación cerrado por invitación, sentando además las bases arquitectónicas para una futura segunda fase de desarrollo enfocada en simulacros de competición y métricas de rendimiento avanzadas.

Fundamentos Estéticos y Lenguaje Visual

El lenguaje visual de DBCWorkspace debe reflejar la naturaleza seria, analítica y profundamente enfocada del entrenamiento para competencias ICPC. Para lograr el estado de "software en calma", la interfaz debe asimilar la estética intelectual del modo oscuro de Obsidian⁵, la densidad hiperfuncional de aplicaciones como Linear⁶ y el brutalismo tabular de AtCoder.⁷ El principio rector de este diseño es que la interfaz jamás debe competir por una atención que no se ha ganado; los elementos estructurales y de navegación deben retroceder visualmente para asegurar que el contenido central (el código, las matemáticas y los datos del problema) mantenga el protagonismo absoluto.⁸

La adopción de un diseño exclusivamente oscuro no es una simple preferencia estilística, sino una decisión ergonómica fundamentada. Durante las prolongadas sesiones de entrenamiento que a menudo se extienden hasta altas horas de la noche, el modo oscuro reduce drásticamente la fatiga visual al disminuir la emisión de luz y el resplandor de la pantalla. El diseño de Obsidian demuestra que un modo oscuro bien ejecutado sirve a la función y no a la moda, reduciendo el ruido visual para que el contenido renderizado en Markdown y los gráficos complejos brillen con claridad.⁵ Del mismo modo, la aplicación Linear utiliza una paleta de colores basada en un negro casi absoluto, denominado "Marketing Black" en su sistema de diseño, el cual se percibe más como un vacío infinito que como una superficie plana.⁶

Para implementar esta estética en DBCWorkspace utilizando Tailwind CSS, se debe establecer un sistema de variables de diseño (tokens) rigurosamente calibrado. La evitación de la fatiga visual requiere que el contraste entre el texto y el fondo sea óptimo sin llegar al extremo del texto blanco puro sobre fondo negro puro, lo cual puede generar un efecto de halo (halation) perjudicial para la legibilidad.

Rol del Token en la	Código	Implementación	Racionalidad y
---------------------	--------	----------------	----------------

Interfaz	Hexadecimal Sugerido	en Tailwind CSS	Experiencia de Usuario (UX)
Fondo Base del Lienzo (Canvas)	#08090a o #0A0A0B	bg-neutral-950 modificado	Proporciona un vacío oscuro profundo inspirado en Linear, minimizando el resplandor general de la pantalla. ⁶
Fondo de Superficie/Tarjetas	#0F172A a #1E293B	bg-slate-900 a bg-slate-800	Crea una elevación sutil para bloques de código y tarjetas de problemas sin necesidad de usar sombras pronunciadas, similar a Obsidian. ⁵
Texto Principal	#EEEEEE	text-gray-100	El texto en un tono gris muy claro previene el efecto de halo y mantiene una alta legibilidad para lecturas prolongadas. ⁵
Texto Secundario o Muteado	#8cabd9	text-slate-400	Utilizado para metadatos como marcas de tiempo y etiquetas menores, estableciendo una clara jerarquía visual. ¹⁰
Acento Primario Interactivo	#A882FF	text-purple-400	El morado característico de Obsidian atrae la atención hacia botones y enlaces activos sin competir

			con el contenido escrito. ⁵
--	--	--	--

La arquitectura de estilos globales en Next.js debe implementar estos colores primarios a nivel del elemento raíz de HTML para evitar cualquier destello de temas incorrectos o parpadeos de contenido sin estilo (Flash of Unstyled Content) durante la carga inicial, un problema común si no se gestionan correctamente las variables CSS.¹¹ Además, la interfaz debe prescindir por completo de gradientes tridimensionales, sombras paralelas agresivas y elementos superpuestos innecesarios, favoreciendo un aspecto plano y ligero que dirija la mirada naturalmente hacia el área del editor y las tablas de datos, que son los actores principales del sistema.¹²

El tratamiento de la tipografía es igualmente crítico para el rendimiento y la legibilidad. La filosofía de diseño de Obsidian destaca por no cargar fuentes web personalizadas, dependiendo enteramente de las fuentes nativas del sistema operativo.⁵ Esta decisión técnica elimina por completo el riesgo de experimentar textos invisibles (Flash of Invisible Text) durante la carga de red y transmite la confianza de que la tipografía nativa de la plataforma es suficientemente buena para el trabajo intelectual.⁵ DBCWorkspace debe adoptar esta misma postura. Para los elementos de la interfaz general (menús, botones, encabezados), se debe utilizar la familia tipográfica sin remates del sistema mediante la clase font-sans de Tailwind (que recurrirá a San Francisco en macOS, Segoe UI en Windows y Roboto en Android/ChromeOS). Sin embargo, para las áreas críticas de contenido, como los bloques de código, las soluciones y las demostraciones algorítmicas, es imperativo el uso de una fuente monoespaciada de alta legibilidad. La clase font-mono garantizará que los operadores matemáticos, las variables y las estructuras de datos mantengan una alineación vertical perfecta. En el contexto del modo oscuro, los diseñadores expertos recomiendan aumentar ligeramente el espaciado entre letras (tracking) y la altura de línea (leading) para abrir las formas de las letras, ya que el texto claro sobre fondos oscuros tiende a parecer visualmente más denso; por tanto, el uso de utilidades como leading-relaxed en Tailwind es esencial para los párrafos extensos de las editoriales.¹⁰

La estructura y separación de los elementos en la pantalla debe basarse en el principio de que la estructura debe sentirse, pero no verse.⁸ En lugar de permitir que las líneas divisorias proliferen innecesariamente y saturen la plataforma, la interfaz debe redondear bordes y suavizar contrastes.⁸ Un patrón de diseño deficiente en el desarrollo de tablas de datos es el uso excesivo de bordes de cuadrícula que compiten visualmente con la información.⁹ DBCWorkspace debe apoyarse en el espacio en blanco (whitespace) estratégico y en separadores horizontales extremadamente sutiles, utilizando clases como border-b border-border last:border-0 con una opacidad muy baja (por ejemplo, border-slate-800), logrando una estructura limpia que permita que los propios datos definan la geometría del espacio, de manera similar a como GitHub gestiona sus tablas densas de incidencias (issues).⁴

Arquitectura de Navegación y Disposición Espacial

La disposición de los elementos en DBCWorkspace debe facilitar una interacción de alta

frecuencia. El entrenamiento para ICPC requiere que el equipo alterne constantemente entre listas de problemas, visualización de soluciones editoriales y cuadernos de anotaciones matemáticas. Por lo tanto, la navegación debe fundamentarse en la memoria espacial, permitiendo a los tres usuarios moverse por la aplicación de manera puramente habitual, casi sin necesidad de enfocar la vista en los menús.

El patrón arquitectónico óptimo para este nivel de densidad es el diseño de barra lateral de doble columna (dual-column sidebar layout), una estructura ampliamente adoptada en cuadros de mando empresariales y aplicaciones de software como servicio (SaaS) complejas.⁹ Utilizando la biblioteca de componentes Shadcn/UI, construida sobre Radix UI y Tailwind CSS, es posible implementar primitivas robustas como SidebarProvider, Sidebar y SidebarRail para orquestar este comportamiento de manera accesible y coherente en Next.js.¹⁵

La arquitectura de la barra lateral se divide en dos zonas interactivas. El riel primario, ubicado en el extremo izquierdo de la pantalla, debe funcionar como un menú de navegación global compuesto únicamente por iconos (Problemas, Editoriales, Roadmap/Simulacros, Cuaderno de Algoritmos, Configuración).¹⁴ Al interactuar con uno de estos iconos, un panel secundario más ancho se despliega para revelar la navegación contextual.¹⁴ Por ejemplo, al seleccionar la sección de "Problemas", el panel secundario mostrará una lista jerárquica de etiquetas y categorías (como "Programación Dinámica", "Algoritmos de Grafos", "Estructuras de Datos"), permitiendo un filtrado rápido inspirado en la categorización de AtCoder.¹⁶ Este enfoque de doble columna maximiza la eficiencia del espacio horizontal, permitiendo que el área de contenido principal (el editor de soluciones o la tabla de problemas) ocupe la mayor parte de la pantalla, al tiempo que mantiene la profundidad de navegación fácilmente accesible a un solo clic de distancia.

Para garantizar una experiencia de usuario ininterrumpida, el estado de colapso de la barra lateral debe ser gestionado a través de un contexto de React y persistido localmente utilizando cookies o el almacenamiento local del navegador (localStorage).¹⁷ Esto asegura que si un usuario contrae el panel secundario para concentrarse en la redacción de un algoritmo complejo, la aplicación recordará esa preferencia incluso si el usuario recarga la página o cierra la sesión temporalmente.¹⁷ Adicionalmente, aplicando una técnica refinada observada en el rediseño de Linear, la barra lateral puede atenuarse sutilmente (disminuyendo su opacidad o contraste) cuando el usuario enfoca su atención en el área de contenido principal, reduciendo su prominencia visual y asegurando que la visión periférica del programador no se vea perturbada por elementos de navegación inactivos.⁸

Diseño y Ergonomía de Tablas de Alta Densidad (CRUD de Problemas)

El núcleo operativo de DBCWorkspace en su primera fase de desarrollo es el sistema de gestión, o CRUD (Crear, Leer, Actualizar, Borrar), de problemas de programación competitiva. La visualización de este inventario exige el diseño de una tabla de datos implacablemente densa que sea capaz de rastrear múltiples variables simultáneamente: el nombre del problema, las etiquetas asociadas, el nivel de dificultad (EASY, MEDIUM, HARD, UNRATED) y el estado de

resolución específico del equipo (no resuelto, resuelto de manera individual, o resuelto en equipo).

En el ámbito del diseño de interfaces para usuarios avanzados y herramientas profesionales, la densidad no es un defecto de diseño, sino una característica fundamental.⁹ Los usuarios que interactúan con grandes volúmenes de datos a diario desprecian el exceso de espacio en blanco que los obliga a desplazarse interminablemente. La tabla debe construirse utilizando los componentes semánticos nativos de HTML (table, thead, tbody, tr, th, td) integrados con las variantes de Shadcn/UI y el motor de utilidades de Tailwind CSS, lo cual garantiza de antemano el soporte para accesibilidad mediante lectores de pantalla y navegación por teclado.⁹

Para lograr una tabla que sea escaneable visualmente en fracciones de segundo, el diseño debe adherirse estrictamente a un conjunto de reglas ergonómicas de alineación de datos ⁹:

1. **Alineación a la izquierda para datos cualitativos:** El texto descriptivo, como los títulos de los problemas y los bloques de etiquetas (tags), debe alinearse sistemáticamente a la izquierda, coincidiendo con el patrón de lectura occidental.⁹
2. **Alineación a la derecha para datos cuantitativos:** Los valores numéricos, tales como el tiempo invertido en la resolución, el número de intentos fallidos o las métricas de rendimiento futuras, deben alinearse invariablemente a la derecha. Esta alineación por valor posicional permite que el ojo humano compare magnitudes de un solo vistazo sin esfuerzo cognitivo.⁹
3. **Alineación central para estados lógicos:** Los indicadores visuales de estado, como las insignias de dificultad o los avatares que indican quién resolvió el problema, deben centrarse para crear un eje vertical predecible.¹⁹

Un desafío común en las tablas de alta densidad es el desbordamiento de texto. Un título de problema inusualmente largo no debe envolverse en múltiples líneas, ya que esto destruye el ritmo vertical de la tabla y dificulta el escaneo rápido.⁹ La solución técnica consiste en aplicar la utilidad truncate de Tailwind para asegurar que el texto se limite a una sola línea terminada en puntos suspensivos, emparejada con un componente interactivo de tipo Tooltip (información sobre herramientas) de Shadcn/UI que revele la cadena de texto completa al pasar el cursor (hover) o al enfocar con el teclado.⁹

Para empoderar a los tres miembros del equipo con opciones de visualización personalizadas, la cabecera de la tabla debe integrar un control de densidad (Density Toggle) utilizando el componente ToggleGroup de Shadcn/UI, ofreciendo tres modos espaciales: Compacto, Cómodo y Espacioso.²⁰ El modo "Compacto" es crucial para el análisis masivo de problemas; en este estado, el relleno interno (padding) de las celdas se reduce a su mínima expresión (por ejemplo, px-2 py-1) y el tamaño de la fuente se ajusta a text-xs, permitiendo encajar docenas de problemas en una sola pantalla sin necesidad de interacción física con la rueda del ratón.⁹ Independientemente de la densidad elegida, la cabecera de la tabla (thead) debe poseer un comportamiento pegajoso (sticky header) mediante la clase sticky top-0, garantizando que los títulos de las columnas permanezcan anclados y visibles en la parte superior del contenedor incluso cuando el usuario desciende hacia el fondo de un largo historial de entrenamiento.⁹

La codificación visual de la dificultad y el estado de los problemas es un área donde DBCWorkspace debe tomar inspiración directa de la interfaz gráfica de AtCoder, conocida por

su clasificación implacable y clara.³ En lugar de depender exclusivamente de texto para comunicar la dificultad, el sistema debe emplear insignias (badges) o círculos de progreso coloreados que el cerebro pueda procesar preatentivamente (antes del pensamiento consciente).⁷

Nivel de Dificultad ICPC	Esquema de Color Inspirado en AtCoder	Implementación Típica de Insignia en Tailwind
EASY (Fácil)	Tonos Cian/Verde Azulado	text-cyan-400 bg-cyan-950/50 border border-cyan-800
MEDIUM (Medio)	Tonos Azules Intensos	text-blue-400 bg-blue-950/50 border border-blue-800
HARD (Difícil)	Tonos Naranjas/Amarillos	text-orange-400 bg-orange-950/50 border border-orange-800
UNRATED (Sin Clasificar)	Tonos Grises Neutros	text-slate-400 bg-slate-800/50 border border-slate-700

El estado de resolución presenta una complejidad adicional. Dado que el equipo ICPC tiene una composición cerrada y estática (Cristóbal, Briana y David), la tabla no necesita sistemas abstractos de usuarios. El estado "resuelto de manera individual" puede representarse elegantemente mediante una matriz compacta de avatares superpuestos o indicadores de puntos con las iniciales de cada miembro. Si un usuario ha resuelto el problema, su inicial se ilumina con el color de acento primario; si no, permanece en un gris atenuado. El estado cumbre, "resuelto en equipo", debe trascender los avatares individuales y sustituirse por una iconografía unificada, como una estrella dorada sólida o el escudo del equipo, proporcionando un refuerzo visual inmediato de las victorias colaborativas.

Documentación Algorítmica y la Experiencia del Editor Matemático

El propósito fundamental de DBCWorkspace no es únicamente rastrear qué problemas se han completado, sino capturar el razonamiento abstracto y la metodología empleada para resolverlos. La documentación de soluciones individuales y editoriales consolidadas es el proceso donde la práctica bruta se cristaliza en conocimiento algorítmico retenido.

Cuando un miembro del equipo redacta una solución, la interfaz debe capturar una arquitectura de datos multifacética: el código fuente (en lenguajes como C++, Java o Python), el razonamiento matemático y estratégico, y los metadatos numéricos (el tiempo invertido en segundos/minutos y el número total de envíos o intentos). Presentar todos estos campos de entrada simultáneamente puede generar un entorno de edición caótico. Para mantener la concentración, la interfaz debe adoptar una disposición donde los campos de metadatos se alojen de manera compacta en un panel de propiedades lateral o en una barra de estado superior, liberando la inmensa mayoría de la pantalla para el lienzo del editor de texto enriquecido, imitando así el entorno sereno de los editores de código profesionales. La captura del razonamiento algorítmico exige un soporte impecable y fluido para la notación matemática mediante LaTeX. Para este propósito, la integración de Tiptap en el ecosistema de Next.js 14 es la decisión arquitectónica más robusta.²¹ Tiptap es un entorno de trabajo sin cabeza (headless framework) construido sobre el poderoso motor ProseMirror, lo que significa que proporciona toda la lógica de edición de texto enriquecido, gestión de nodos y seguimiento de estado interno sin imponer ninguna interfaz de usuario o estilo predeterminado.²¹ Esto otorga un control absoluto a los desarrolladores para aplicar la estética del sistema de diseño oscuro mediante las utilidades de Tailwind CSS. Sin embargo, el despliegue de Tiptap dentro de una arquitectura basada en el App Router de Next.js 14 conlleva desafíos específicos de renderizado. Dado que Next.js emplea el renderizado del lado del servidor (Server-Side Rendering o SSR) por defecto, la hidratación de un editor de texto complejo que depende en gran medida del estado del navegador y del Document Object Model (DOM) puede desencadenar errores de discrepancia de hidratación (hydration mismatch errors) en React.²² Para sortear este obstáculo técnico, la configuración de la instancia del editor mediante el gancho `useEditor` debe incluir forzosamente la propiedad `immediatelyRender: false`. Esta directiva suspende la inicialización visual del editor durante la fase de procesamiento en el servidor, garantizando que el componente interactivo se ensamble de manera segura y exclusiva en el cliente una vez que la hidratación de React haya concluido exitosamente.²² El motor subyacente para el renderizado de la tipografía matemática es KaTeX, una biblioteca de alto rendimiento elegida por su velocidad de procesamiento superior en comparación con alternativas como MathJax. La conexión entre Tiptap y KaTeX se establece mediante extensiones matemáticas específicas (como `@tiptap/extension-mathematics` o repositorios de código abierto optimizados como `aarkue/tiptap-math-extension`).²⁴ La ergonomía de esta integración es lo que define una experiencia de usuario de clase mundial:

- **Transición Fluida entre Edición y Renderizado:** El flujo de trabajo ideal requiere que al momento de que el usuario envuelva una fórmula con el símbolo de dólar (por ejemplo, $O(N \log N)$ para matemáticas en línea, o $\$$ para bloques matemáticos estructurales), el editor asigne instantáneamente un nodo interno.²⁵ Mientras el cursor parpadeante permanezca dentro de los límites del nodo matemático, el sistema debe mostrar el código fuente crudo de LaTeX, permitiendo la edición sin restricciones. En el milisegundo en que el usuario mueve el cursor fuera del nodo (evento de pérdida de foco o blur), el texto crudo debe transformarse mágicamente en la salida tipográfica renderizada de

KaTeX.²⁵

- **Gestión de Errores Indulgente:** En el fragor del entrenamiento, los errores tipográficos al escribir macros de LaTeX son inevitables. Es vital configurar el motor con el parámetro `throwOnError: false` en el objeto de configuración `katexOptions`.²⁴ De este modo, si un usuario teclea accidentalmente un comando inválido (como `\farc{1}{2}` en lugar de `\frac`), el editor no lanzará una excepción catastrófica que rompa la interfaz, sino que renderizará el texto erróneo en un color de alerta (como el rojo `#ef4444` de Tailwind) acompañado de un cuadro de información (tooltip) que detalle el error de sintaxis, permitiendo una corrección rápida y sin estrés.²⁷
- **Aceleración mediante Macros:** Para optimizar la velocidad de escritura del equipo, la configuración del editor debe predefinir macros personalizadas para la notación recurrente en ICPC. Comandos cortos para conjuntos numéricos (por ejemplo, mapear `\R` a `\mathbb{R}`) o estructuras algorítmicas frecuentes acelerarán sustancialmente la documentación del razonamiento.²⁴

La estilización de los componentes generados por Tiptap y KaTeX bajo el esquema del modo oscuro se simplifica inmensamente gracias al uso del plugin oficial `@tailwindcss/typography`. Al aplicar la clase `prose-invert` al contenedor principal del editor, Tailwind intercepta automáticamente todas las etiquetas HTML nativas generadas por ProseMirror (encabezados, listas, bloques de citas y bloques de código) y adapta sus colores y contrastes a la escala de grises oscuros del sistema, logrando una legibilidad prístina sin necesidad de escribir hojas de cascada (CSS) manuales y propensas a errores.²⁸

Finalmente, el flujo de trabajo de la documentación distingue claramente entre la solución redactada por un individuo y la "Editorial". La editorial actúa como la fuente unificada y canonizada de la verdad, a menudo sintetizada tras una sesión de retrospectiva post-mortem, mientras que las soluciones individuales muestran el código y enfoque particular de Cristóbal, Briana o David. Para evitar la sobrecarga cognitiva en la pantalla de visualización del problema, esta información no debe apilarse verticalmente de manera infinita. En su lugar, la interfaz de usuario debe emplear un componente de pestañas (Tabs de Shadcn/UI).²⁹ La primera pestaña mostrará invariablemente la Editorial consolidada, y las pestañas subsecuentes aislarán las soluciones individuales de cada miembro. Esta segmentación permite a un usuario leer la explicación teórica general y luego saltar directamente a la pestaña de David para analizar cómo él, específicamente, implementó una estructura de Árbol de Segmentos para ese mismo desafío, preservando el contexto y la claridad mental.

Gestión de Estado URL, Búsqueda Avanzada y Filtros

Con una base de datos de PostgreSQL que, previsiblemente, albergará cientos o miles de problemas de programación competitiva a lo largo de los ciclos de entrenamiento, la capacidad de "encontrabilidad" (findability) referenciada en el hexágono de UX de Morville se convierte en la métrica definitiva de éxito.¹

El principal error en el diseño de interfaces modernas de una sola página (Single Page Applications o SPA) es aislar el estado de los filtros en variables locales de React (como `useState`). Si la configuración de búsqueda de un usuario (por ejemplo, buscar problemas de

Grafos de dificultad Difícil que no han sido resueltos) reside únicamente en la memoria del navegador, la experiencia se rompe si el usuario recarga la página o intenta compartir el estado actual copiando el enlace a un compañero de equipo a través de un mensaje directo. Para resolver este paradigma en Next.js 14, la arquitectura debe sincronizar el estado de la interfaz gráfica directamente con los parámetros de búsqueda de la URL (URL search parameters) como la única fuente de la verdad, implementando para ello la biblioteca especializada `nuqs`.³⁰ `nuqs` proporciona un gestor de estado con seguridad de tipos que actualiza la barra de direcciones del navegador en tiempo real de manera reactiva. Este enfoque presenta dos ventajas arquitectónicas fundamentales:

1. **Regulación de Tráfico y Rendimiento:** Los campos de entrada HTML, como una barra de búsqueda de texto libre, emiten actualizaciones a una frecuencia sumamente alta mientras el usuario escribe. Empujar cada pulsación de tecla a la API de Historial (History API) del navegador puede desencadenar límites de tasa de petición (rate limits), los cuales son particularmente restrictivos en navegadores como Safari, resultando en una interfaz congelada o con saltos bruscos.³⁰ La biblioteca `nuqs` gestiona esta complejidad internamente, acumulando y limitando (throttling) las solicitudes de actualización a la URL, lo que garantiza una experiencia de escritura fluida y libre de interrupciones mientras el estado alcanza la consistencia en segundo plano.³⁰
2. **Filtrado Optimizado desde el Servidor:** Al elevar el estado de búsqueda a la URL (generando direcciones estructuradas como `?tag=DP&diff=HARD&status=unsolved`), los Componentes de Servidor (React Server Components) de Next.js 14 pueden interceptar la propiedad `searchParams` antes de que el código llegue al navegador del cliente.³¹ Esto permite ejecutar consultas altamente optimizadas hacia la base de datos a través de Prisma desde el propio servidor, saltándose la necesidad de renderizar indicadores de carga (spinners) en el lado del cliente y entregando una tabla de datos completa, filtrada e hidratada de manera casi instantánea.³¹

Para satisfacer las demandas de usuarios con perfiles técnicos avanzados, la caja de búsqueda principal debe trascender el modelo básico y adoptar una sintaxis de consulta lógica inspirada en los filtros avanzados de GitHub.³³ Más allá de hacer clic en menús desplegados, los miembros del equipo deben poder escribir consultas semánticas complejas directamente, tales como `is:unsolved difficulty:HARD tag:DP,Math`. El analizador léxico (parser) del sistema debe estar programado para interpretar comas dentro de un mismo campo como un operador lógico 'O' (OR) —donde `tag:DP,Math` recuperará problemas etiquetados con Programación Dinámica o Matemáticas— y procesar los espacios entre distintos campos como un operador lógico 'Y' (AND).³⁴

Para aquellos momentos en los que la selección visual mediante clics sea preferible a la escritura de comandos, la barra de búsqueda se complementará con una interfaz de filtros facetados utilizando selectores emergentes (Command Palettes de Shadcn/UI). La genialidad UX de este enfoque dual es que, al seleccionar un filtro visualmente desde un menú, el sistema inyecta dinámicamente la sintaxis de texto correspondiente en la barra de búsqueda, educando implícitamente al usuario sobre el poder de los comandos de teclado a través de la propia interacción.²⁹ Cualquier filtro activo debe mostrarse en la interfaz como una etiqueta removible

(chip o badge) flotando claramente sobre la tabla de datos, garantizando que el usuario comprenda instantáneamente qué condiciones están restringiendo el conjunto de problemas que está visualizando.⁹

Arquitectura de Autenticación, Seguridad y Gestión de Invitaciones

DBCWorkspace opera bajo una filosofía estricta de sistema cerrado. Al ser una herramienta privada destinada a albergar el intelecto competitivo de solo tres personas específicas, el registro público debe estar clausurado herméticamente. La topología de seguridad se apoya en el robusto ecosistema de Auth.js (anteriormente NextAuth.js en su versión 5) manejando sesiones a través de JSON Web Tokens (JWT), con la integración de Prisma conectada a la base de datos PostgreSQL de Neon.

Dado que las bibliotecas estándar de autenticación no incluyen de manera nativa flujos de "solo invitación" (invite-only) sin integraciones de correo electrónico, la arquitectura requiere diseñar una implementación personalizada construida alrededor del proveedor de Credentials (Credenciales) de Auth.js y un sistema criptográfico de generación de tokens de un solo uso.³⁶ Las especificaciones exigen que no haya envío automático de correos; por lo tanto, el acceso se distribuye exclusivamente a través de la copia manual de enlaces generados.

El ciclo de vida del flujo cerrado de invitaciones opera a través de los siguientes mecanismos en la base de datos y la interfaz de usuario:

1. **Generación Segura (Acción del Administrador):** Cristóbal, asumiendo el rol de ADMIN, navega hacia un panel de control protegido mediante el middleware de Next.js. A través de una acción de servidor (Server Action), invoca la generación de una nueva invitación. El servidor en la nube de Neon, a través de Prisma, inserta un registro en una tabla dedicada de Invitaciones. Este registro contiene un token criptográfico único (generado mediante identificadores universales o módulos criptográficos nativos de Node.js), un identificador de rol predeterminado (por ejemplo, MEMBER), una marca de tiempo de expiración rigurosa y un estado booleano de consumido.
2. **Distribución y Enrutamiento:** El sistema expone el token generado a través de la interfaz de administrador en formato de enlace estructurado (ej. `dbcworkspace.com/invite?token=xyz123`). El administrador copia el enlace al portapapeles y lo transmite de forma externa y segura a Briana o David (vía mensajería encriptada).
3. **Intercepción y Registro Personalizado:** Cuando el receptor accede al enlace, la página de destino extrae el token de la URL mediante los parámetros de búsqueda. Se presenta al usuario un formulario de registro que solicita la definición de un correo electrónico y una contraseña segura. Al enviar el formulario, el proceso es capturado por la función de devolución de llamada (callback) `authorize` del proveedor de Credentials de Auth.js.³⁶
4. **Transacción de Base de Datos y Creación de Sesión:** Dentro del bloque `authorize`, el servidor ejecuta una transacción atómica utilizando Prisma. Primero, consulta la tabla de Invitaciones para validar matemáticamente que el token coincide, que su fecha de caducidad es vigente y que no ha sido consumido previamente.³⁶ Si la validación

criptográfica fracasa, la transacción se aborta y se rechaza la autenticación. Si es exitosa, el servidor aplica una función de derivación de claves pesada (como bcrypt) para generar un hash seguro de la contraseña proporcionada por el usuario.³⁷

Inmediatamente, se crea un nuevo registro en la tabla de Usuarios de PostgreSQL, asignándole el rol definido en la invitación original. Finalmente, el registro de la invitación se marca de manera permanente como consumido para clausurar cualquier intento de reutilización maliciosa, y se devuelve el objeto de usuario a Auth.js para iniciar la sesión.³⁶

Para optimizar el rendimiento general del sistema y evitar latencias derivadas de consultar la base de datos en cada navegación de ruta, el paradigma de sesión debe basarse enteramente en tokens JWT asimétricos almacenados en cookies seguras HTTP-only del navegador, evitando las sesiones gestionadas en base de datos.³⁸

La estrategia de JWT exige enriquecer el payload del token durante el ciclo de vida de la autenticación. Utilizando los callbacks de jwt y session en la configuración global de Auth.js, el identificador único del usuario y su rol crítico (ADMIN o MEMBER) se inyectan permanentemente dentro del token.⁴⁰ Esta inyección de metadatos permite que las políticas de Control de Acceso Basado en Roles (RBAC) se ejecuten en tiempo real.

La seguridad de las rutas se centraliza en el archivo middleware.ts característico de Next.js 14. Este middleware actúa como un muro de contención en el borde (edge) de la aplicación, interceptando cada petición HTTP dirigida a rutas protegidas. Al inspeccionar el payload decodificado de la cookie JWT, el middleware garantiza que usuarios anónimos sean inmediatamente redirigidos al portal de inicio de sesión.⁴¹ Adicionalmente, evalúa las reglas RBAC impidiendo, por ejemplo, que los usuarios con rol de MEMBER accedan a los end-points analíticos o de generación de tokens reservados exclusivamente para la ruta /admin. Un desafío arquitectónico en este nivel radica en la compatibilidad de Prisma con el entorno Edge en el que corre el middleware; dado que Prisma depende de binarios nativos no soportados nativamente en Edge, las consultas directas a la base de datos desde el middleware deben evitarse o gestionarse mediante Prisma Accelerate o adaptadores de controladores específicos (como @prisma/adapter-neon), confiando en su defecto en la verificación criptográfica autosuficiente del token JWT para la autorización del enrutamiento.⁴³

Proyección y Escalabilidad: Hoja de Ruta de la Fase 2

Aunque la implementación inicial (Fase 1) establece la columna vertebral del CRUD de problemas y la edición algorítmica, las decisiones arquitectónicas de la base de datos relacional y los patrones de interfaz de usuario deben estar meticulosamente preparados para soportar la hoja de ruta de la Fase 2, la cual introduce mecánicas punitivas y analíticas avanzadas.

Transformación de UI para Modo de Competición (Contest Mode)

La futura implementación del "Modo Simulacro" con reglas oficiales de la ICPC altera radicalmente los requisitos ergonómicos de la plataforma. La ICPC impone un entorno reglado de alta presión: tiempos límite estrictos, una penalización matemática de 20 minutos por cada envío incorrecto previo a la solución exitosa, y el infame "congelamiento" del marcador (freeze) durante la hora culminante de la competencia.

La interfaz de usuario del Modo Competición debe abandonar el diseño contemplativo y centrado en la documentación prolongada a favor de una ergonomía reactiva y de alto estrés:

- **Señalización Tensión-Visual (Freeze):** La interfaz debe comunicar inconfundiblemente el cambio de estado del sistema. Cuando se active el período de "congelamiento" del marcador, la interfaz gráfica debe responder con una transición temática agresiva; por ejemplo, el sutil color de acento morado primario podría cambiar globalmente a un tono azul gélido (ice-blue) o a un rojo estricto, informando al subconsciente de los competidores que la dinámica del sistema ha entrado en fase crítica, sin alterar la estructura fundamental del diseño.
- **Telemetría de Penalizaciones en Tiempo Real:** Durante el simulacro, el registro de envíos interactuará con una bitácora en vivo. A diferencia de las evaluaciones estándar, cualquier evaluación de código que resulte en estado incorrecto (Wrong Answer, Time Limit Exceeded) desencadenará una alerta visual en la interfaz, parpadeando sutilmente en colores de advertencia (rojo o naranja) y registrando de manera prominente la sumatoria acumulada de penalización (por ejemplo, +20, +40 minutos).
- **Arquitectura de Módulos Post-Mortem:** Tras la conclusión de un concurso y el levantamiento del congelamiento, el diseño prevé una transición automatizada hacia una vista Post-Mortem. Aquí, las herramientas analíticas tomarán protagonismo mediante la implementación de gráficos de línea de tiempo interactivos. Este componente visual permitirá mapear los momentos exactos de los envíos a lo largo de las 5 horas del concurso, identificando cuellos de botella temporales y permitiendo al equipo anotar sus procesos de depuración (debugging) de manera colaborativa, cruzando los datos crudos con el razonamiento retrospectivo.

El Sistema de Calificación Personalizada (Rating sin Elo)

La especificación del sistema descarta explícitamente el uso de algoritmos de emparejamiento como el sistema Elo, el cual está diseñado para evaluar la probabilidad de victoria en enfrentamientos de suma cero (jugador contra jugador). En su lugar, el sistema de DBCWorkspace medirá el crecimiento personal y grupal ponderando el percentil del tiempo de resolución frente al universo histórico del equipo, la dificultad intrínseca del problema y la penalización absoluta por intentos fallidos.

Para viabilizar este modelo estadístico, el esquema del modelo relacional de Prisma en la Fase 1 debe trazar una bitácora quirúrgica. Cada intento de solución documentado no será simplemente una entrada de texto, sino un vector de datos que encapsulará obligatoriamente un sello temporal de creación (`created_at`), un número entero preciso que represente la duración exacta de la ventana de resolución en segundos (`time_invested`), y el conteo incremental de envíos al juez (`submission_count`).

La representación visual (UX) de este sistema de calificación personalizado es un terreno psicológico delicado. Para evitar la toxicidad inherente a las tablas de clasificación hipercompetitivas dentro de un equipo tan pequeño y unido, el sistema omitirá los marcadores agresivos. En su lugar, las métricas se visualizarán como un gráfico de crecimiento longitudinal y personal. El cálculo algorítmico interno normalizará el tiempo invertido en función del peso de la dificultad del reto; por ejemplo, si un problema está catalogado como HARD, el tiempo de

resolución esperado o tolerable será estadísticamente mayor, minimizando la penalidad en la fórmula interna del servidor. La penalización por intentos fallidos restará puntos mediante una constante punitiva de erosión, reflejando fielmente el rigor de la competición oficial. La interfaz deberá desglosar este cálculo matemático mediante cuadros de herramientas (tooltips) descriptivos al posar el cursor sobre la puntuación, promoviendo un ambiente de total transparencia donde el usuario comprenda con exactitud qué variables de su metodología de programación modularon su métrica de desempeño, en lugar de recibir un número abstracto indescifrable.

El Cuaderno de Algoritmos y el Registro de Cambios (Changelog)

Finalmente, el Cuaderno de Algoritmos (Notebook) funcionará como el compendio arcano del conocimiento de la triada. Mientras que la vista CRUD de Problemas fragmenta la información en ejercicios aislados e históricos, el Notebook es un repositorio estructural para plantillas genéricas reutilizables (implementaciones puras de un Árbol de Segmentos, el Algoritmo de Tarjan para componentes fuertemente conexos, o Árboles de Fenwick).¹⁶ La experiencia de usuario en este módulo imitará casi al milímetro el funcionamiento de la aplicación Obsidian: una topología de carpetas anidadas desestructuradas en el panel lateral de navegación, frente a un expansivo lienzo de edición Tiptap a pantalla completa, dependiendo de las macros de KaTeX para demostrar la corrección y las complejidades de tiempo y espacio de los algoritmos de manera teórica antes de su aplicación en los concursos.

El Registro de Cambios (Changelog) complementa esta etapa avanzada asegurando la comunicación asíncrona técnica. A medida que Cristóbal implementa expansiones o correcciones de errores en DBCWorkspace, es esencial documentar estas iteraciones para el equipo. La aproximación UX para este requisito se materializará mediante un componente deslizante lateral (como un Sheet de Shadcn/UI), accesible desde un icono no intrusivo en el riel de navegación principal. Este panel emergente presentará una lista cronológica, renderizada a partir de archivos Markdown (MD), detallando minuciosamente las nuevas funcionalidades o ajustes en el algoritmo de calificación, garantizando que el equipo opere siempre sobre una base de entendimiento mutuo sobre las capacidades vigentes de su entorno de entrenamiento privado, sin saturar la pantalla principal ni interferir en el ciclo de codificación pura.

Obras citadas

1. 2285158488147046.pdf, <https://drive.google.com/open?id=1UfQOJYcglfytNseaLGy0WoZ0kj-GPBm>
2. Appearance - Obsidian Help, fecha de acceso: julio 5, 2026, <https://obsidian.md/help/appearance>
3. AtCoder Problems - kenkoooo.github.io, fecha de acceso: julio 5, 2026, <https://kenkoooo.com/atcoder/>
4. Customizing the table layout - GitHub Docs, fecha de acceso: julio 5, 2026, <https://docs.github.com/en/issues/planning-and-tracking-with-projects/customizing-views-in-your-project/customizing-the-table-layout>
5. Obsidian: Dark Intellectual Composability for Knowledge Workers, fecha de acceso: julio 5, 2026, <https://blakecrosley.com/guides/design/obsidian>

6. soul-design-md/designs/linear/DESIGN.md at main · soulcore-dev/soul-design-md - GitHub, fecha de acceso: julio 5, 2026, <https://github.com/soulcore-dev/soul-design-md/blob/main/designs/linear/DESIGN.md>
7. AtCoder Statistics - GitHub, fecha de acceso: julio 5, 2026, <https://github.com/wulukewu/atcoder-statistics>
8. A calmer interface for a product in motion - Linear, fecha de acceso: julio 5, 2026, <https://linear.app/now/behind-the-latest-design-refresh>
9. Data table UI design reference guide for 2026 | Setproduct Blog, fecha de acceso: julio 5, 2026, <https://www.setproduct.com/blog/data-table-ui-design>
10. Dark Mode in CSS - CSS-Tricks, fecha de acceso: julio 5, 2026, <https://css-tricks.com/dark-modes-with-css/>
11. prefers-color-scheme: Hello darkness, my old friend | Articles - web.dev, fecha de acceso: julio 5, 2026, <https://web.dev/articles/prefers-color-scheme>
12. UI refresh: a modern light and dark theme · Issue #2114 - GitHub, fecha de acceso: julio 5, 2026, <https://github.com/eclipse-platform/eclipse.platform.ui/issues/2114>
13. 6+ Shadcn Table Components - Creative Tim UI, fecha de acceso: julio 5, 2026, <https://www.creative-tim.com/ui/docs/components/table>
14. React Dual Column Sidebar Block - shadcn.io, fecha de acceso: julio 5, 2026, <https://www.shadcn.io/blocks/sidebar-dual-column>
15. Sidebar - Shadcn UI, fecha de acceso: julio 5, 2026, <https://ui.shadcn.com/docs/components/radix/sidebar>
16. AtCoder Categories, fecha de acceso: julio 5, 2026, <https://atcoder-categories.github.io/>
17. Sidebar Navigation Blocks based on shadcn/ui - Creative Tim UI, fecha de acceso: julio 5, 2026, <https://www.creative-tim.com/ui/blocks/application-admin/sidebar>
18. Shadcn Table - Base UI and Radix UI, fecha de acceso: julio 5, 2026, <https://shadcnstudio.com/docs/components/table>
19. Building production ready data tables with shadcn/ui - shadcncraft Blog, fecha de acceso: julio 5, 2026, <https://shadcncraft.com/blog/building-production-ready-data-tables-with-shadcn-ui>
20. React Table Density Toggle Component - shadcn.io, fecha de acceso: julio 5, 2026, <https://www.shadcn.io/blocks/tables-density>
21. Style your editor | Tiptap Editor Docs, fecha de acceso: julio 5, 2026, <https://tiptap.dev/docs/editor/getting-started/style-editor>
22. Next.js | Tiptap Editor Docs, fecha de acceso: julio 5, 2026, <https://tiptap.dev/docs/editor/getting-started/install/nextjs>
23. React | Tiptap Editor Docs, fecha de acceso: julio 5, 2026, <https://tiptap.dev/docs/editor/getting-started/install/react>
24. Mathematics extension | Tiptap Editor Docs, fecha de acceso: julio 5, 2026, <https://tiptap.dev/docs/editor/extensions/nodes/mathematics>
25. aarkue/tiptap-math-extension: Math Extension for the TipTap Editor - GitHub, fecha de acceso: julio 5, 2026, <https://github.com/aarkue/tiptap-math-extension>

26. KaTeX Integration #179 - ueberdosis/tiptap - GitHub, fecha de acceso: julio 5, 2026, <https://github.com/ueberdosis/tiptap/issues/179>
27. Options - KaTeX, fecha de acceso: julio 5, 2026, <https://katex.org/docs/options>
28. Effortless Typography, Even in Dark Mode - Tailwind CSS, fecha de acceso: julio 5, 2026, <https://tailwindcss.com/blog/tailwindcss-typography-v0-5>
29. Data Table 14 - Multi-Filter Data Table - Shadcnblocks.com, fecha de acceso: julio 5, 2026, <https://www.shadcnblocks.com/block/data-table14>
30. How to update query param in Next js 14 app directory like - Stack Overflow, fecha de acceso: julio 5, 2026, <https://stackoverflow.com/questions/77896076/how-to-update-query-param-in-next-js-14-app-directory-like-input-type-text-on>
31. How to properly manage search params in NextJS App router. Leverage the power of nuqs the right way | by Jaime Ayala | Medium, fecha de acceso: julio 5, 2026, <https://medium.com/@Jaimayal/how-to-properly-manage-search-params-in-nextjs-app-router-leverage-the-power-of-nuqs-the-right-way-9f7238cff76a>
32. Managing search parameters in Next.js with nuqs - LogRocket Blog, fecha de acceso: julio 5, 2026, <https://blog.logrocket.com/managing-search-parameters-next-js-nuqs/>
33. Filtering and searching issues and pull requests - GitHub Docs, fecha de acceso: julio 5, 2026, <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/filtering-and-searching-issues-and-pull-requests>
34. Filtering projects - GitHub Docs, fecha de acceso: julio 5, 2026, <https://docs.github.com/en/issues/planning-and-tracking-with-projects/customizing-views-in-your-project/filtering-projects>
35. Can I search github labels with logical operator OR? - Stack Overflow, fecha de acceso: julio 5, 2026, <https://stackoverflow.com/questions/29136057/can-i-search-github-labels-with-logical-operator-or>
36. Built-in invite-only authentication in NextAuth · nextauthjs next-auth · Discussion #4106, fecha de acceso: julio 5, 2026, <https://github.com/nextauthjs/next-auth/discussions/4106>
37. Building a Secure Registration Flow with Prisma, MongoDB, and NextAuth in Next.js, fecha de acceso: julio 5, 2026, <https://medium.com/@josephmuhindo089/building-a-secure-registration-flow-with-prisma-mongodb-and-nextauth-in-next-js-17fdb0195986>
38. Guides: Authentication - Next.js, fecha de acceso: julio 5, 2026, <https://nextjs.org/docs/pages/guides/authentication>
39. Implementing JWT-Based Authentication with Next.js v14 and NextAuth v4 | by Kcsanjeeb, fecha de acceso: julio 5, 2026, <https://medium.com/@kcsanjeeb091/implementing-jwt-based-authentication-with-next-js-v14-and-nextauth-v4-e3efca4b158b>
40. Callbacks | NextAuth.js, fecha de acceso: julio 5, 2026, <https://next-auth.js.org/configuration/callbacks>

41. Authentication and Authorization with Next.js 14: JWT || COOKIES || MONGODB - YouTube, fecha de acceso: julio 5, 2026,
<https://www.youtube.com/watch?v=fvYA6g1Q7ml>
42. Best way to handle JWT in Next.js 14 with a separate Express backend : r/nextjs - Reddit, fecha de acceso: julio 5, 2026,
https://www.reddit.com/r/nextjs/comments/1fjt5dy/best_way_to_handle_jwt_in_nextjs_14_with_a/
43. Prisma adapter - Auth.js, fecha de acceso: julio 5, 2026,
<https://authjs.dev/getting-started/adapters/prisma>
44. Complete Guide to Authentication with Auth.js, Next.js, Prisma, and Neon - Medium, fecha de acceso: julio 5, 2026,
<https://medium.com/@radhemugdal/complete-guide-to-authentication-with-auth-js-next-js-prisma-and-neon-7dcf1a946468>